



Interacting With My RAG Chatbot in the Terminal



Natasha Ong

<https://www.linkedin.com/in/natasha-ong/>

```
us-east-2 +
[cloudshell-user@ip-10-130-20-225 ~]$ aws bedrock-agent-runtime retrieve-and-generate --input '{"text": "What is NextWork?"}' --retrieve-and-genera
te-configuration '{
  "knowledgeBaseConfiguration": {
    "knowledgeBaseId": "VJQWVGEXFO",
    "modelArn": "arn:aws:bedrock:us-east-2::foundation-model/meta.llama3-3-70b-instruct-v1:0"
  },
  "type": "KNOWLEDGE_BASE"
}'
{
  "citations": [
    {
      "generatedResponsePart": {
        "textResponsePart": {
          "span": {
            "end": 146,
            "start": 0
          },
          "text": "NextWork is an organization that provides projects and resources for learning and development, with the goal of helping peop
le find jobs they love."
        }
      }
    }
  ]
}
```



Introducing Today's Project!

In this project, we will set up a RAG chatbot and interact with it over the terminal using CLI commands. We're doing this project to learn how to interact with our tools programmatically - instead of relying on the console.

Tools and concepts

The services we used were S3, Bedrock (Knowledge Base), OpenSearch Serverless, AWS CloudShell, and the key concepts we learnt included running Bedrock commands, finding IDs using commands/the console, how to update an entire KBase over the terminal.

Project reflection

This project took us approximately two hours including demo time. The most challenging part was finding the ModelARN to go into the retrieve-and-generate command. It was most rewarding to see our KBase answers questions on new info in CloudShell.



Natasha Ong
NextWork Student

NextWork.org

We did this project today to level up our experience and familiarity with RAG chatbots, CLI commands + managing a Knowledge Base. This project let us do all three in a very hands on way. We're one step closer towards integrating our chatbot in an app



Setting Up The Knowledge Base

To set up our Knowledge Base, we used S3 to store the documents (i.e. information) we want our RAG chatbot to know. The documents we uploaded contain information about NextWork projects!

We also created a Knowledge Base in Bedrock to process and understand the documents that we've uploaded. Later on, when we ask our chatbot a question, it's the KBase that will retrieve the relevant information! It uses embeddings and a vector store.

Our chatbot needs access to two AI models, which were Titan Text Embeddings (to create embeddings that represent the meaning of my documents), and Llama 3.3 Instruct (to convert the KBase's results into chat responses). We then sync'd KBase <> S3.

The screenshot shows the 'Review and create' step in the AWS Bedrock console. On the left, a vertical progress bar lists four steps: Step 1 (Provide Knowledge Base details), Step 2 (Configure data source), Step 3 (Select embeddings model and configure vector store), and Step 4 (Review and create). Step 4 is currently selected and highlighted with a blue circle. The main content area is titled 'Review and create' and 'Step 1: Provide details'. It contains a table with the following information:

Knowledge Base details	
Knowledge Base name nextwork-rag-documentation	Knowledge Base description This Knowledge Base stores all documentation at NextWork.
Service role AmazonBedrockExecutionRoleForKnowledgeBase_dt08k	Knowledge base type Knowledge base use vector store
Data source type S3	Log Deliveries —

On the right side of the console, there are two circular icons: an information icon (i) and a refresh icon (circular arrow). Below these is an 'Edit' button.



Running CLI Commands in CloudShell

AWS CLI is a software that lets us interact with our AWS resources/services using commands instead of clicks (i.e. the Management Console). To start testing CLI commands, we first opened CloudShell, which is AWS's browser-based terminal.

When we first ran a Bedrock command, we ran into an error because we needed to provide values for the Knowledge Base's ID and the AI model's ARN.

While finding the parameters takes extra time, the advantage of using the CLI is the ability to test different setups i.e. different Knowledge Bases and models much faster. We can swap out parameter values instead of clicking around in the console.

```
us-east-2 +  
[cloudshell-user@ip-10-130-20-225 ~]$ aws bedrock-agent-runtime retrieve-and-generate \  
> --input '{"text": "What is NextWork?"}' \  
> --retrieve-and-generate-configuration '{  
>   "knowledgeBaseConfiguration": {  
>     "knowledgeBaseId": "your_knowledge_base_id",  
>     "modelArn": "your_model_arn"  
>   },  
>   "type": "KNOWLEDGE_BASE"  
> }'  
  
An error occurred (ValidationException) when calling the RetrieveAndGenerate operation: 3 validation errors  
detected: Value 'your_knowledge_base_id' at 'retrieveAndGenerateConfiguration.knowledgeBaseConfiguration.k  
nowledgeBaseId' failed to satisfy constraint: Member must have length less than or equal to 10; Value 'your  
_knowledge_base_id' at 'retrieveAndGenerateConfiguration.knowledgeBaseConfiguration.knowledgeBaseId' failed  
to satisfy constraint: Member must satisfy regular expression pattern: [0-9a-zA-Z]+; Value 'your_model_arn  
' at 'retrieveAndGenerateConfiguration.knowledgeBaseConfiguration.modelArn' failed to satisfy constraint: M  
ember must satisfy regular expression pattern: (arn:aws(-[^\:]+)?:(bedrock|sagemaker):[a-z0-9-]{1,20}:([0-9]
```



Running Bedrock Commands

To find the required values, we had to visit our KBase's page from the console, and run a separate CLI command to find the ModelArn. The Bedrock command ran successfully and showed us a chat response for "What is NextWork?" within the terminal!

The retrieve-and-generate command typically also outputs all the raw text and metadata related to our Knowledge Base. To tidy up the terminal response, we added `--query` parameters that gets the terminal to ONLY output the chatbot's text response.

```
us-east-2 +  
[cloudshell-user@ip-10-130-20-225 ~]$ aws bedrock-agent-runtime retrieve-and-generate --input '{"text": "What is NextWork?"}' --retrieve-and-genera  
te-configuration {  
  "knowledgeBaseConfiguration": {  
    "knowledgeBaseId": "VJQWVGEXF0",  
    "modelArn": "arn:aws:bedrock:us-east-2::foundation-model/meta.llama3-3-70b-instruct-v1:0"  
  },  
  "type": "KNOWLEDGE_BASE"  
},  
{  
  "citations": [  
    {  
      "generatedResponsePart": {  
        "textResponsePart": {  
          "span": {  
            "end": 146,  
            "start": 0  
          },  
          "text": "NextWork is an organization that provides projects and resources for learning and development, with the goal of helping peop  
le find jobs they love."  
        }  
      }  
    }  
  ]  
}
```



Extending Your Knowledge Base

In a project extension, we asked our Knowledge Base about one of the lovely viewers/commenters of this live project demo (called Timtimol AI Sutdio). We noticed that our Knowledge Base had no information about them, so it couldn't provide a response.

To add new information to our Knowledge Base, we ran commands to create a text file, upload that to an S3 bucket and start an ingestion job (syncing). Compared to using the console, this process was faster although you do have to find lots of IDs.

To validate the update worked, we ran a retrieve-and-generate command that asks our chatbot about the new information ("Who is Timtimol AI Studio?"). The Knowledge Base successfully returned the information that we uploaded to our data source.

```
us-east-2 +
[cloudshell-user@ip-10-130-20-225 ~]$ aws bedrock-agent get-ingestion-job \
> --knowledge-base-id "VJQWVGEXFO" \
> --data-source-id "W7DFGRJZQG" \
> --ingestion-job-id "6RBP2WNQBP"
{
  "ingestionJob": {
    "dataSourceId": "W7DFGRJZQG",
    "ingestionJobId": "6RBP2WNQBP",
    "knowledgeBaseId": "VJQWVGEXFO",
    "startedAt": "2025-02-13T22:54:07.823997+00:00",
    "statistics": {
      "numberOfDocumentsDeleted": 0,
      "numberOfDocumentsFailed": 0,
      "numberOfDocumentsScanned": 11,
      "numberOfMetadataDocumentsModified": 0,
      "numberOfMetadataDocumentsScanned": 0,
      "numberOfModifiedDocumentsIndexed": 0,
      "numberOfNewDocumentsIndexed": 0
    }
  }
}
```



Interacting with AI Models Directly

On top of chatting with my chatbot, we also interacted directly with an AI model via the terminal by running the bedrock-runtime invoke-model command. This bypasses our Knowledge Base and answers general questions!

```
[cloudshell-user@ip-10-130-20-225 ~]$ aws bedrock-runtime invoke-model \
> --model-id meta.llama3-3-70b-instruct-v1:0 \
> --body '{"prompt": "Write a short poem about cats"}' \
> --cli-binary-format raw-in-base64-out \
> --region us-east-2 \
> --output text \
> /dev/stdout

{"generation": "\nHere is a short poem about cats:\n\nWhiskers twitch, eyes shine bright,\nFur as soft as moonlight night.\nPurrs rumble deep, a soothing sound,\nAs they curl up, snug and round.\n\nWith claws that grasp and ears so fine,\nThey rule the house, a feline shrine.\nIndependent souls, yet loving too,\nOur feline friends, a joy to pursue.", "prompt_token_count": 7, "generation_token_count": 84, "stop_reason": "stop"}
[cloudshell-user@ip-10-130-20-225 ~]$
```



NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

